Neural Machine Translation of Rare Words with Subword Units

Sennrich, Haddow and Birch (ACL 2016), arXiv:1508.07909

Paper summary for the CS-781 capstone

The paper at a glance

Citation

R. Sennrich, B. Haddow, A. Birch. *Neural Machine Translation of Rare Words with Subword Units*. ACL 2016. University of Edinburgh.

Why it matters for our capstone

BPE is how our Phase-1 tokenizer is built (`scripts/02_train_tokenizer.py`). Every model in all seven phases reads BPE tokens.

One-sentence summary

Adapt the byte-pair-encoding compression algorithm to split words into frequent subword units, so a fixed-size vocabulary can represent *any* word.

Where it appears in our slides

Main deck: “Step 02: a legal byte-level BPE tokenizer”. Glossary: “Tokenization” section.

Roadmap

1. The problem
2. The basic idea: byte-pair encoding
3. Experiments
4. Assessment and outlook

Translation is an open-vocabulary problem

- Neural models use a **fixed vocabulary**, typically 30,000–50,000 words.
- Real text keeps producing **new words**: names, compounds, inflections, loanwords.
- Words outside the vocabulary become `UNK`.
- Earlier fix: a **back-off dictionary**. Translate `UNK` by looking the source word up, or copy it.

Problems with back-off:

- Words do not map one-to-one across languages.
- The model cannot *generate* unseen words.
- Copying fails when alphabets differ, e.g. English to Russian.

The paper's example

German *Abwasser|behandlungs|anlage* means "sewage water treatment plant". One long compound, three meaningful parts. A single fixed-length vector for the whole word is a poor fit; its parts are what translate.

A legal example

Vakalatnama, *Panchnama*, *Mukhtarnama* share the suffix *-nama* ("document"). A word-level vocabulary may miss the rarer ones entirely.

Key intuition: rare words are made of translatable pieces

Many word types can be translated by a competent translator even if new, from known smaller units:

- **Names**: copy or transliterate. *Barack Obama* becomes Russian *Barak Obama* (in Cyrillic).
- **Cognates and loanwords**: regular character changes. *claustrophobia*, *Klaustrophobie*.
- **Morphologically complex words**: translate the parts. *solar system*, *Sonnen|system*.

Evidence from the paper

100 rare German tokens (outside the top 50,000) were analysed:

Category	Count
Compounds	56
Names	21
Loanwords, common origin	6
Transparent affixation	5
Number / code identifier	2

Most rare words are translatable from their parts.

The trade-off: vocabulary size vs sequence length

- **Words**: short sequences, but a huge vocabulary and unknown words.
- **Characters**: tiny vocabulary and no unknowns, but very long sequences. Attention must span far more positions.
- **Goal**: a compact vocabulary *and* short text, with no unknown words.

Earlier segmenters from statistical MT, such as compound splitting and Morfessor, only moderately shrink the vocabulary and still leave unknown words.

German training data (about 100M tokens):

Segmentation	Tokens	Types	UNK
none (words)	100 m	1,750,000	1079
characters	550 m	3,000	0
char. bigrams	306 m	20,000	34
compound splitting	102 m	1,100,000	643
Morfessor	109 m	544,000	237
BPE	**112 m**	**63,000**	**0**
BPE (joint)	111 m	82,000	32

UNK = unknown tokens in newstest2013.

BPE makes text only 12% longer than words, with a vocabulary 28× smaller and **no unknowns**.

From compression to segmentation

Original BPE (Gage, 1994) is a data-compression method: repeatedly replace the most frequent pair of bytes with a new, unused byte.

Adaptation for words:

1. Write every word as characters plus an end-of-word marker `</w>`.
2. Count all adjacent symbol pairs, weighted by word frequency.
3. Merge the most frequent pair (A, B) into a new symbol AB everywhere.
4. Repeat for a chosen number of merges.

Vocabulary = characters + merges; the **number of merges is the only hyperparameter**.

Design details

- Pairs never cross word boundaries, so BPE can run on a word-frequency dictionary, which is fast.
- The `</w>` marker lets the original text be restored exactly.
- Frequent words end up as one symbol; rare words as several pieces.
- Unlike Huffman codes, the pieces are *readable* subwords, so the network can reuse them.

Easy example: the paper's toy vocabulary

Word counts: `low`×5, `lower`×2, `newest`×6, `widest`×3.

Step	Merge	Count
1	e + s → es	9
2	es + t → est	9
3	est + </w> → est</w>	9
4	l + o → lo	7
5	lo + w → low	7
6	n + e → ne	6
7	ne + w → new	6
8	new + est</w> → newest</w>	6

Counts: "e s" appears in newest (6) and widest (3), so 9.

Applying the merges to text

After 10 merges the vocabulary gives:

- `newest` → `newest</w>`: one symbol.
- `widest` → `wi d est</w>`.
- `lower` → `low e r </w>`.

A **new** word `lowest`, never seen, becomes `low est</w>`, two known pieces.

Legal flavour

If "appeal", "appellant" and "appellate" are frequent, BPE learns `appe`, `appell`, and so on. The rare *appellee* then splits into known pieces like `appell` + `ee`.

The algorithm in a few lines

```
def get_stats(vocab):                # count adjacent pairs
    pairs = defaultdict(int)
    for word, freq in vocab.items():
        s = word.split()
        for i in range(len(s) - 1):
            pairs[s[i], s[i+1]] += freq
    return pairs

vocab = {'l o w </w>': 5, 'l o w e r </w>': 2,
         'n e w e s t </w>': 6, 'w i d e s t </w>': 3}
for i in range(num_merges):
    pairs = get_stats(vocab)
    best = max(pairs, key=pairs.get)  # most frequent
    vocab = merge_vocab(best, vocab)  # replace everywhere
```

Adapted from the paper's Algorithm 1.

At test time

Split a word into characters, then apply the learned merges *in order*. This works for *any* word. The only possible unknowns are unseen characters.

Cost

Learning is linear in dictionary size per merge. Real implementations index pairs and update counts incrementally.

Joint BPE: consistent pieces in both languages

Two options:

- **Separate BPE**: learn one encoding per language. More compact, and every piece is seen in its own language.
- **Joint BPE**: learn one encoding on the union of both vocabularies. Names are split *the same way* on both sides, so they are easier to copy or transliterate.

For English–Russian, Russian is transliterated to Latin script for learning, and the merges are mapped back to Cyrillic.

Why consistency matters (paper's example)

Name *Mirzayeva*; Russian shown transliterated:

- Separate BPE: `Mirz|ayeva` → `Mir|za|eva`. The pieces do not line up.
- Joint BPE: `Mir|za|yeva` → `Mir|za|eva`. Piece-by-piece transliteration.

Inconsistent splits in training data taught the separate system errors like *rak* → *pra*.

Setup

- **Task**: WMT 2015, English→German (4.2M sentence pairs, about 100M tokens) and English→Russian (2.6M pairs, about 50M tokens).
- **Model**: attention-based RNN encoder–decoder (Bahdanau et al., 2015). Hidden size 1000, embeddings 620.
- **Training**: Adadelta, batch 80, about 7 days per model; beam size 12.
- **Metrics**: BLEU, chrF3 (character n-gram F-score), and unigram F1 split into all, **rare** and **OOV** words.

Systems compared

- **WUnk**: words; unknowns stay `UNK`.
- **WDict**: words plus back-off dictionary (baseline).
- **C2-50k**: 50k frequent words, character bigrams for the rest.
- **BPE-60k**: separate BPE.
- **BPE-J90k**: joint BPE.

No subword system uses a back-off dictionary.

Results: English→German (newstest2015)

System	BLEU single	BLEU ens-8	chrF3 single	chrF3 ens-8	unigram F1 (%), ensemble rare	unigram F1 (%), ensemble OOV
Syntax-based SMT	24.4	–	55.3	–	–	–
WUnk	20.6	22.8	47.2	48.9	20.4	0.0
WDict (baseline)	22.0	24.2	50.5	52.4	36.8	36.8
C2-50k	**22.8**	**25.3**	51.9	53.5	40.5	30.9
BPE-60k	21.5	24.5	52.0	53.9	40.9	29.3
BPE-J90k	**22.8**	24.7	51.7	**54.1**	**41.8**	33.6

- Rare-word unigram F1 rises from 36.8 to **41.8**.
- For English→German, copying unknown names works well, so the dictionary's OOV score stays high.
- Subword ensembles beat the baseline by 0.3–1.1 BLEU on this pair.

Results: English→Russian (different alphabets)

System	BLEU		chrF3		unigram F1 (%), ensemble	
	single	ens-8	single	ens-8	rare	OOV
Phrase-based SMT	24.3	–	53.8	–	–	–
WUnk	18.8	22.4	46.5	49.9	25.2	0.0
WDict (baseline)	19.1	22.8	47.5	51.0	26.5	6.6
C2-50k	**20.9**	**24.1**	49.0	51.6	27.8	17.4
BPE-60k	20.5	23.6	**49.8**	52.7	29.7	15.6
BPE-J90k	20.4	**24.1**	49.7	**53.0**	29.7	**18.3**

- Copying fails across alphabets, so the dictionary gets only 6.6 OOV F1. Subwords reach **18.3**, because they learn to *transliterate*.
- Joint BPE gains 1.3 BLEU and 2.0 chrF3 over the baseline.

Analysis: rare words and what the models produce

By word frequency: all systems are similar on the 50,000 most frequent words. Between ranks 50,000 and 500,000, representing words as *subwords* beats representing them as rare whole words. Rank 50,000 means about 60 training occurrences; rank 500,000 means about 2.

Lesson: subword units are *less sparse* than rare words. A *smaller* vocabulary with more subword splitting can *improve* accuracy.

Translations from the paper

health research institutes → reference *Gesundheitsforschungsinstitute*

- WDict: *Forschungsinstitute* (drops "health").
- BPE-J90k: *Gesundheits|forsch|ungsin|stitute*. Correct, even though the splits ignore morpheme boundaries.

rakfisk → reference *rakfiska* (Cyrillic)

- WDict: copies *rakfisk* unchanged.
- BPE-J90k: *rak|f|iska*. Transliterated correctly.

Connection to our capstone

- Our tokenizer is **byte-level BPE** (the GPT-2 variant): it starts from 256 bytes instead of characters, so even unseen characters cannot be unknown.
- `tokenizer.vocab_size` (default 32,000) sets the number of merges, the paper's only hyperparameter.
- Our **fertility** metric (tokens per word) is the paper's "text size" axis.

Experiments the paper suggests

- Vocabulary size sweep: 16k, 32k, 48k. The paper found a smaller vocabulary can help rare words.
- Legal BPE vs GPT-2's web BPE: which splits legal words better?
- Check rare legal terms and Indian names: how many pieces do they take?
- For NER (Phase 2): do BPE splits break entity boundaries?

Strengths and limitations

Strengths

- Simple, fast, language-independent.
- True open vocabulary with a compact, fixed symbol set.
- One hyperparameter.
- Removes the need for back-off dictionaries.
- Became the default tokenizer family for modern language models.

Limitations

- Greedy and frequency-based: splits often ignore morphology.
- Vocabulary size was chosen "somewhat arbitrarily", by the authors' own account.
- Deterministic segmentation: a word is always split the same way, so the model never sees alternatives.
- Separate BPE can split names inconsistently across languages.
- BLEU and chrF3 disagreed on the best system; gains on rare words are hard to see in aggregate metrics.

Future research directions (1): from the paper and after

Direction	Details
Learn the vocabulary size automatically	Proposed in the paper: the best size should depend on the language pair and data size
Bilingually informed segmentation	Proposed in the paper: create more alignable units, while working without the target text at runtime
Byte-level BPE	GPT-2 (Radford et al., 2019) runs BPE on bytes, so no symbol can ever be unknown
Probabilistic segmentation	Unigram LM tokenizer and SentencePiece (Kudo, 2018; Kudo and Richardson, 2018)
Segmentation as regularisation	BPE-dropout (Provilkov et al., 2020) randomly skips merges during training
Tokenizer-free models	ByT5 (Xue et al., 2022) and CANINE (Clark et al., 2022) read bytes or characters directly

Future research directions (2): open questions

Open in general

- What makes a tokenizer "good" for a model, beyond compression?
- Fairness across languages: the same content can cost several times more tokens in some languages than in English (Petrov et al., 2023).
- Handling numbers and dates consistently.
- Reducing variance between training runs, which the paper flags as an open problem for NMT.

Open for legal NLP

- Keeping citations such as "(2019) 5 SCC 123" and section numbers such as "302" intact or predictably split.
- Indian names and transliterated terms in English judgments.
- Domain-adaptive vocabularies: add legal merges to a general tokenizer.
- Character offsets for NER: subword boundaries must map back to exact characters.

Summary

- Word-level NMT cannot handle an open vocabulary; back-off dictionaries are a brittle patch.
- **BPE**: start from characters and repeatedly merge the most frequent adjacent pair. Frequent words become single units; rare words become known pieces.
- On German data: 63k types instead of 1.75M, text only 12% longer, and no unknown tokens.
- Better rare-word translation (EN→DE rare unigram F1 36.8 to 41.8), transliteration across alphabets, and up to 1.1 (EN→DE) and 1.3 (EN→RU) BLEU.
- Joint BPE keeps segmentations consistent across languages.
- For us: the foundation of the Phase-1 tokenizer, and a source of tokenizer experiments.

Reference

Sennrich, R., Haddow, B., and Birch, A. (2016). Neural Machine Translation of Rare Words with Subword Units. In *Proceedings of ACL 2016*. arXiv:1508.07909.

Other work cited: Gage (1994) A New Algorithm for Data Compression; Bahdanau et al. (2015); Radford et al. (2019) GPT-2; Kudo (2018) Subword Regularization; Kudo and Richardson (2018) SentencePiece; Provilkov et al. (2020) BPE-Dropout; Xue et al. (2022) ByT5; Clark et al. (2022) CANINE; Petrov et al. (2023) Language Model Tokenizers Introduce Unfairness Between Languages.